BIRMINGHAM CITY University

CMP6200/DIG6200

INDIVIDUAL UNDERGRADUATE PROJECT

2024–2025

A3: Dissertation

SOFTWARE SECURITY ANALYSIS USING AI (INTRUSION DETECTION SYSTEMS)

d

Course: Computing and Information Technology
Student Name: Daniel Ofure Akpomon-Daniels
Student Number: 21193928
Supervisor Name: Ambreen Hussain

Abstract

The escalating complexity of cyberattacks requires advanced intrusion detection systems (IDS) beyond traditional signature-based methods. This dissertation explores leveraging artificial intelligence, specifically machine learning techniques, to enhance software security by developing an AI-based IDS framework. Focusing on anomaly detection, the study implemented and evaluated Machine learning models using a contemporary dataset. Rigorous data preprocessing was performed before training. The evaluation employs standard classification metrics, revealed high performance in binary classification. However, multi-class classification performance on rare attack types presented significant challenges for both models. Practical testing integration into a simulated environment was unfortunately hindered by technical issues. The results demonstrate machine learning's potential for effective basic intrusion detection but highlight the critical need to address class imbalance for comprehensive threat type identification and overcome deployment challenges for real-world applicability.

Acknowledgements

First and foremost, I would like to give all glory to God for my accomplishment and for giving me the strength, guidance and perseverance to perform this project.

I am incredibly thankful for the invaluable support and expert guidance provided by my supervisor, Ambreen Hussain. Her insights, encouragement, and constructive feedback were necessary throughout this journey.

My appreciation also goes to Birmingham City University for providing the resources, environment, and opportunity to pursue this academic endeavour.

Finally, I would like to express my heartfelt appreciation to my family and friends. Your unwavering love, patience, and understanding have been my constant source of motivation. This achievement would not have been possible without you.

Table of Contents

Abstract ii
Acknowledgements iii
Glossary vi
List of Figures vii
List of Tables viii
1 Introduction 1
1.1 Problem Definition 1
1.2 Scope 2
1.3 Rationale 2
1.4 Project Aim and Objectives 3
2 Literature Review 3
2.1 Themes 3
2.2 Review of Literature 4
2.2.1 Machine learning and Deep learning Algorithms for Intrusion Detection 4
2.2.2 Hybrid AI Models for Enhanced Detection 4
2.2.3 AI Based Incident Response Integration 5
2.2.4 Scalability and Real-Time Processing 6
2.2.5 Adversarial Resilience 7
2.3 Theory 7
3 Method and Implementation 8
3.1 Methodology 8
3.2 Tools and Environment 9
3.3 Design and Development 9
3.3.1 Data Preprocessing and Feature Engineering 9
3.3.2 Model Selection 12
3.4 Implementation 12
3.4.1 Training Process 12
3.4.2 Cross-Validation 13
4 Evaluation 14
4.1.1 Evaluation Metrics 14
4.1.2 Dataset 15
4.1.3 Baseline Systems 15
4.1.4 Limitations of the Evaluation 16

4.1.5 Testing Approach 16
4.2 Results 17
4.2.1 Binary Classification Performance 17
4.2.2 Multi-class Classification Performance 18
4.2.3 Comparative Summary 20
4.2.4 Challenges in Practical Testing Implementation 20
4.3 Discussion 21
5 Conclusions 23
6 Recommendations for future work 24
7 References 25
8 Appendices 29
8.1 Gantt Chart 29

Glossary

IDS	Intrusion Detection Systems
KNN	K-Nearest Neighbour
CNN	Convolutional Neural Networks
IoT	Internet of Things
TF-IDF	Term Frequency Inverse Document Frequency
DCNN	Deep Convolutional Neural Networks
LSTM	Long Short Term Memory networks
LIME	SHapley Additive exPlanations
SOC	Security Operations Centers
SVM	Support Vector Machine
SAE	Stacked AutoEncoder
FNN	Feed-forward Neural Networks
SNN	Self-normalising Neural Networks
SIEM	Security Information and Event Management

List of Figures

Figure 1.Agile Iterative Sprint Structure. (GeeksforGeeks, 2023) 9
Figure 2. Data Normalisation of the Dataset's numerical values. 10
Figure 3. Categorical Encoding - One Hot Encoding. 10
Figure 4. Prepared Dataset for Binary Classification. 11
Figure 5. Prepared Dataset for Multi-class Classification. 11
Figure 6. Pie chart Distribution of Binary Labels 11
Figure 7. Pie chart Distribution of Multi-Class Labels 12
Figure 8. KNN Binary Classification Training 12
Figure 9. Random Forest Binary Classification Training 13
Figure 10. KNN Multi-class Classification Training 13
Figure 11. Random Forest Multi-class Classification Training 13
Figure 12. Unsuccessful API Connection on Wazuh's Dashboard 21

List of Tables

Table 1. Binary Classification Performance on Dataset Test – ATTACK 17
Table 2. Binary Classification Performance on Dataset Test – NORMAL 17
Table 3. Multi-Class Classification Performance on Dataset Test Set - Weighted Avg. 18
Table 4. Multi-Class Categorical Classification Performance- KNN 19
Table 5. Multi-Class Categorical Classification Performance- Random Forest 19

1 Introduction

The increasing complexity of cyberattacks has created a critical need for advanced intrusion detection systems (IDS) to safeguard software and networks. This report explores an approach to address this challenge using AI-based Intrusion Detection Systems (AI-IDS).

Modern society's increasing reliance on digital systems has fundamentally altered software security. As these systems become more complex and interconnected, they offer a larger attack surface for malicious actors. Traditional signature-based IDS, which rely on known attack patterns, struggle to detect new and evolving threats. This is compounded by the sheer volume of network data which overwhelms human analysts.

1.1 Problem Definition

The persistent digitalisation of modern society has irreversibly transformed the landscape of software security. As software systems become increasingly complex and interconnected, the attack surface expands exponentially, rendering traditional signature-based intrusion detection systems (IDS) inadequate. These systems reliant on pre-defined attack patterns, struggle to identify novel and complex threats, leaving critical infrastructure and sensitive data vulnerable. (Dataguard Insights, 2024) This problem is further intensified by the sheer volume of network traffic and system logs, exceeding the capacity of human analysts to effectively monitor and respond in real-time. (Saha and Pritom, 2023)

The origin of this problem lies in the fundamental change from static, perimeter-based security to dynamic, adaptive threat landscapes. The rise of cloud computing, microservices architectures, and the Internet of Things (IoT) has introduced a new pattern of distributed and temporary systems. Traditional security models, designed for centralised environments, fail to address the inherent complexity of these modern systems.(Cyber Sniper, 2024) Consequently, organisations face a growing risk of data breaches, service disruptions, and financial losses due to undetected intrusions.

The significance of this problem is particularly severe within industries reliant on continuous operations and data integrity, such as finance, healthcare, and critical infrastructure. For example, in the financial sector, a single successful intrusion can lead to substantial financial losses and reputational damage. Similarly, in healthcare, compromised patient data can have severe ethical and legal effects. Furthermore, the positive development of IoT devices, often with limited security capabilities, presents a significant attack vector for malicious actors, potentially impacting essential services and public safety. (Armstrong, 2021)

This dissertation project is motivated by observations within the cybersecurity consulting industry, where we frequently encountered clients struggling with the limitations of existing IDS solutions. Specifically, a recurring challenge involved the inability to detect zero-day exploits and polymorphic malware, which evade traditional signature-based detection. This led to a critical need for more intelligent and adaptive security systems.

The core problem is the deficiency of current intrusion detection systems in effectively addressing the evolving threats and complexities of modern software environments. Traditional methods are unable to cope with the scale, speed, and complexity of contemporary attacks, leading to a critical gap in security strength. (Diana et al., 2025) This gap demands the development and deployment of advanced intrusion detection systems capable of real-time analysis, anomaly detection, and predictive threat modelling.

To address this challenge, this report explores the application of artificial intelligence (AI) techniques, specifically machine learning and deep learning, to enhance intrusion detection capabilities. By leveraging the power of AI, it aims to develop a novel IDS framework that can learn from vast amounts of data, identify subtle anomalies, and adapt to emerging threats. This research seeks to provide a more robust and proactive approach to software security, ultimately contributing to a more resilient and secure digital ecosystem.

1.2 Scope

The scope includes the development of an AI-IDS framework capable of analysing network traffic and system logs to identify anomalies indicative of malicious activity.

The project will primarily investigate supervised and unsupervised learning approaches for anomaly detection. Supervised learning models will be trained on labeled datasets of normal and malicious activity to classify new data, while unsupervised learning will focus on identifying deviations from established normal behavior in unlabeled data. Deep learning techniques, such as neural networks, will be explored for their ability to automatically extract relevant features from raw data and improve detection accuracy.

The research will also address the challenges of detecting zero-day exploits and polymorphic malware, which are not recognisable by traditional signature-based IDS. The goal is to develop an AI-IDS that can generalise from learned patterns to identify novel attacks.

The project scope is bounded by the following:

- **Focus on Network and System-Level Intrusions:** This research will concentrate on detecting intrusions at the network and host levels. It will not extend to the analysis of application-specific vulnerabilities or social engineering attacks. This is justified by the fact that network and system-level intrusions often serve as the initial stages of more complex attacks, and their early detection is crucial for mitigating potential damage (Bishop, 2018).
- **Emphasis on Anomaly Detection:** While misuse detection (detecting known attack patterns) can benefit from AI, this project will primarily emphasise anomaly detection. This is because anomaly detection offers a greater potential for identifying novel and unknown threats, which are a significant challenge for traditional IDS
- **Evaluation in Simulated Environments:** Due to the sensitive nature of real-world network data and the ethical considerations involved, the AI-IDS framework will be evaluated using publicly available datasets and simulated network environments. This approach allows for controlled experimentation and performance evaluation without compromising the security of live systems. While real-world testing is ideal, the use of simulated environments is a widely accepted practice in intrusion detection research

1.3 Rationale

This research addresses the critical gap in effective intrusion detection within modern software security landscapes. The project's genesis stems from observed limitations within the cybersecurity consulting industry, where clients struggle to defend against these advanced attacks.

The application of AI, specifically machine and deep learning, offers a promising avenue for enhancing intrusion detection capabilities. (Mohale and Obagbuwa, 2025) This dissertation aims to develop a robust, adaptive AI-IDS framework capable of real-time anomaly detection and predictive threat modeling.

The benefits extend beyond academic contribution. Organisations reliant on data integrity and continuous operations, such as financial institutions and healthcare providers, will directly benefit from improved security posture. Moreover, the broader cybersecurity community gains insights into effective AI-driven intrusion detection, contributing to a more resilient digital ecosystem. The project's findings will aid in developing more proactive security solutions, ultimately reducing the risk of data breaches and service disruptions.

1.4 Project Aim and Objectives

The project aim is to improve software security by developing a method to efficiently detect and prevent cyberattacks. Using AI techniques, It will be able to create models that can analyse network behaviors and identify both known and unknown attacks.

Project Objectives

- Conduct a comparative analysis on both Machine Learning and Deep learning algorithms, evaluating their performance on standard datasets.
- Develop and test a hybrid approach combining two algorithms for improved detection accuracy aiming for a 5% increase over existing methods.
- Create attack scenarios for software durability and effectiveness testing.
- Implement the framework on an already existing supported software.
- Compile a dissertation report summarising the research findings including performance metrics and insights gained overall proposing a methodology for future research in AI-based IDS.

2 Literature Review

Before starting the project and putting it into action, it is important to understand it from the research literature, approaches, tools used and challenges faced. This process will aid in recognising and fixing any issues ultimately paving the way to finding a resolution.

2.1 Themes

This project will concentrate on several key topics in AI-based intrusion detection systems (IDS). These themes define the scope and subject areas for a literature review, ensuring a deep understanding of the methodology, technologies, and results achieved.

- Machine and Deep learning Algorithms for Intrusion Detection
- Hybrid AI Models for Enhanced Detection
- AI Based Incident Response Integration
- Scalability and Real-Time Processing
- Adversarial Resilience

2.2 Review of Literature

2.2.1 Machine learning and Deep learning Algorithms for Intrusion Detection

Machine learning and deep learning algorithms in Intrusion Detection Systems focus on tackling the complexities presented by modern cyber threats effectively. Traditional Security Information and Event Management (SIEM) systems frequently struggle to cope with attacks and hence require adaptive machine learning approaches. Farooq et al (2018) presented methods to improve IDS such as using a Random Forest Classifier with Term Frequency Inverse Document Frequency (TF-IDF) for text analysis and employing K Means clustering to organise data effectively alongside utilising linear regression to detect unusual behavioral patterns. Moreover One Class Support Vector Machine (SVM) supported by kernel functions including Gaussian, Linear, Polynomial and Radial are being investigated for anomaly detection with the linear kernel showing reliable outcomes in identifying genuine anomalies. When we apply these methods to real world scenarios outlined in the MITRE ATTACK framework we discover that the collective design of Random Forest enhances forecasting precision whereas Decision Tree models result in increased occurrences (Farooq et al., 2018).

Furthermore, A study conducted by Karatas et al., (2018) focuses on using deep learning in IDS addressing the need for advanced detection techniques in increasingly complex networks. A survey of deep learning IDS methods, with an emphasis on datasets like KDD Cup99, reveals that traditional data is widely used, though new datasets could improve detection in future implementations.

Different algorithms, like Random Forest and Logistic Regression along with machine learning and deep learning models such as CNNs (Convolutional Neural Networks) DCNNs (Deep Convolutional Neural Networks) and LSTMs (Long Short Term Memory networks) are often compared for their performance in tasks. Studies indicate that while these methods are effective for detecting anomalies and analysing behavior patterns there are still difficulties in handling imbalanced data and dealing with long term dependencies, in network traffic data. (Varaprasad et al., 2024)

2.2.2 Hybrid AI Models for Enhanced Detection

Hybrid AI models are designed to improve anomaly based Intrusion Detection Systems. It aims to enhance interpretability and decrease alarms in network security using an AI framework with the NSL KDD dataset containing 67,343 normal instances and 58,630 attack instances in the training set. (Ramyavarshini et al., 2022) By combining deep learning methods with tools such as LIME (Local Interpretable Model Agnostic Explanations) and SHAP (SHapley Additive exPlanations) to a system, it provides insights into how models make decisions aiding cybersecurity experts in their work.

The initial steps involve converting category based elements into values and standardising data using Scikit Learn before separating it into training and testing sets with an 80-20 ratio. Visualisation tools like LIME plots and summary plots aid in making the results more understandable for security experts by showcasing how the model detects risks. By utilising these aids such as waterfall plots and force plots in Security Operations Centers (SOC), professionals can enhance their confidence in the models predictions and make informed decisions based on the insights provided. Sriram et al (2022) research findings suggest that using a combination of methods effectively maintains a balance, between accuracy in detecting threats and providing insights into potential security risks, within the network.

The study made by Mohammed I. Alghamdi (2022) introduces an approach to enhance intrusion detection systems (IDS) for IoT environments. The model known as PO-CFNN integrates a political optimiser with a cascade forward neural network to improve detection accuracy and efficiency. He made use of a methodology consisting of three key processes: preprocessing the network data, utilising the CFNN for intrusion identification and classification and applying the PO algorithm for parameter optimisation. Results on a benchmark dataset revealed that the PO-CFNN technique outperformed existing methods, demonstrating superior accuracy and efficiency in detecting various types of attacks.

This highlights the potential of hybrid AI models in addressing security challenges in IoT networks and suggests that future research could explore metaheuristic algorithms for enhanced feature selection.

2.2.3 AI Based Incident Response Integration

A study conducted by Dinesh Chirra (2023) on an AI-driven automated cybersecurity incident response system designed to enhance threat detection and mitigation processes through machine learning algorithms and realtime analytics. The study employs a multi-faceted methodology using simulated attack scenarios and real-world network traffic data for analysis. The research utilised supervised learning with 70% of data for training and 30% for testing which evaluates models on accuracy, precision, recall, and F1-score metrics. The researchers key findings revealed that the Random Forest model achieved 94% accuracy and 96% precision in threat detection while the AI-driven system significantly reduced response times to 5 minutes compared to traditional methods which is 45 minutes. The Isolation Forest model demonstrated a 92% anomaly detection rate but had limitations which included potential biases from historical data reliance and the need for validation in multi-cloud environments.

Additionally, when exploring the integration of artificial intelligence with infrastructure protection to improve threat detection and incident response in cloud environments. Zhang (2020) proposed a plan that utilises AI tools such as machine learning and natural language processing along with data analysis for immediate threat assessment and automated response mechanisms. The methodology used focuses on blending AI based solutions into security structures and Security Information and Event Management (SIEM). This will simplify the process of identifying threats and reacting swiftly to them. The system utilises supervised and unsupervised learning methods in its framework. Supervised models tackle identified risks while unsupervised models aim to identify risks to enhance detection capacities.

The results of the study indicate that by incorporating AI into the system detected an increase in identifying threats by marking more than 85% of unusual activities while also decreasing the number of inaccurate alerts by around 40%. Moreover automating procedures led to a decrease of than 50% in responding to incidents promptly. Despite these advancements challenges such as concerns about data privacy the comprehensibility of AI algorithms and the need for adaptation of models to changing threats are emphasised (Ahmad, 2020). These difficulties highlight the complexities involved in implementing AI for safeguarded infrastructure security.

2.2.4 Scalability and Real-Time Processing

The paper authored by Danyil et al., (2023) on Real-Time Ransomware Detection integrates Extended Berkeley Packet Filter (eBPF), Machine Learning (ML) and Natural Language Processing (NLP). The methodology involves real-time monitoring through eBPF, which collects granular data on system behavior. Machine Learning algorithms are then applied to identify patterns indicative of ransomware, while NLP improves detection by examining text patterns in system information. The system was tested with artificial and actual data sets.

The results show that the detection accuracy stands at 94.7% with precision reaching 92.3% and recall at 93.5%. The average time taken to detect activities was 2.3 seconds, showing that it possess quick response capabilities. However Valerii et al., (2023) observed difficulties dealing with scalability during high volume scenarios, highlighting areas that need improvement. The study signifies a leap in real time ransomware detection methods presenting a more efficient strategy for combatting the changing landscape of cyber threats.

A paper on a project by Herrero et al. (2011) called "RT-MOVICAB-IDS" which addresses real-time intrusion detection integrates Artificial Neural Networks (ANN) and Case-Based Reasoning (CBR) within a Multi-Agent System (MAS) framework to enhance real-time processing and scalability in network security. The system emphasises the need for fast responses to intrusions and addresses time-bounded deliberative processes. Results show its efficiency in CPU utilisation and execution time, showcasing improved responsiveness. Overall, RT-MOVICAB-IDS represents a significant advancement in intrusion detection technology, effectively managing the complexities of modern network environments.

Additionally, in a novel scalable intrusion detection system (IDS) that utilises deep learning techniques to enhance the detection of network intrusions, particularly in the context of big data. Mighan et al., (2020) presents a significant advancement in scalable (IDS) by leveraging deep learning techniques to enhance real-time detection of network intrusions, particularly within the framework of big data. They highlight the advantages of deep learning in managing complex high-dimensional datasets and its capacity to generalise across diverse network environments, making it particularly suitable for real-time applications. The methodology is structured around three core components: data preprocessing, latent feature extraction via a stacked autoencoder (SAE) and attack detection using a Support Vector Machine (SVM) classifier. The SAE plays a crucial role in extracting relevant features from the data, effectively reducing dimensionality and optimising processing efficiency, which is essential for real-time performance. The SVM then classifies these features to identify intrusions promptly.

Results from experiments conducted indicate that the SAE–SVM framework significantly outperforms traditional SVM classifiers in terms of precision and processing speed. This integration of deep learning for feature extraction with machine learning for classification not only enhances diagnostic accuracy but also minimises error rates. Kahani et al., (2020) findings highlight the potential of combining deep learning with big data processing frameworks to improve the scalability and effectiveness of real-time intrusion detection systems which addresses the challenges posed by large-scale network traffic data.

2.2.5 Adversarial Resilience

Addressing the vulnerabilities of machine learning (ML) algorithms in cybersecurity, particularly focusing on adversarial attacks that can deceive Intrusion Detection Systems. Pawlick (2020) investigated four evasion attack methods: Fast Gradient Sign Method, Basic Iterative Method, Carlini and Wagner attack while using the CICIDS2017 dataset to train an Artificial Neural Network (ANN). The methodology used included gathering responses during testing for adversarial examples before training five distinct machine learning classifiers such as Random Forest and Nearest Neighbour to identify these adversarial samples.

The study results highlight the possibility of creating defences against these 4 attacks in machine learning programs and stresses the importance of conducting studies in this field to improve cybersecurity systems resilience against threats from adversaries.

Similarly, Ibitoye et al., (2019) analysed adversarial attacks against deep learning-based Intrusion Detection Systems (IDS) within Internet of Things (IoT) networks focusing on the ability of adversarial examples to mislead machine learning models into producing inaccurate predictions. Their methodology involves implementing IDS through two deep learning architectures: Feed-forward Neural Networks (FNN) and Self-normalising Neural Networks (SNN). They evaluated the resilience of these models using three adversarial sample generation techniques: the Fast Gradient Sign Method (FGSM), Basic Iteration Method (BIM), and Projected Gradient Descent (PGD).

The study's findings reveal that although the SNN demonstrated a 9% higher performance over the FNN in resisting adversarial samples, both systems recorded accuracy rates below 50% when subjected to adversarial conditions indicating considerable vulnerability. Additionally, Shafiq (2019) noticed that while feature normalisation improved the overall performance metrics, it unexpectedly increased both systems vulnerability to adversarial inputs. These results underscore the challenge of designing resilient IDS models capable of withstanding adversarial manipulation effectively.

2.3 Theory

This theory outlines the operational framework of AI-IDS, detailing the design methods, algorithms, and processes involved in enhancing software security.

Operational Framework of AI-IDS

1. Algorithm Design and Implementation

   Machine Learning Algorithms: Various machine learning algorithms will be used for classification and anomaly detection. Implemented algorithm include:

   - Random Forest: Used for its robustness and ability to handle high-dimensional data and also its effective classification of known attack patterns.
   - K-nearest Neighbor: This is an algorithm that is used for classification and regression. It predicts data labels by finding the "k" closest labelled data points and using their majority class or average value.

   Deep Learning Models: Advanced models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory networks (LSTMs) will be used because of their ability to learn complex patterns in large datasets. CNNs are particularly effective for spatial data, while LSTMs can excel in sequential anomaly detection, making them suitable for time-series intrusion detection.

2. Combining Techniques: We will be combining Random Forest for initial classification with an LSTM for sequential anomaly detection can provide a comprehensive analysis of network behavior thereby enhancing the detection accuracy

3. Automated Response Mechanisms: The IDS will be integrated with Security Information and Event Management (SIEM) systems to automate incident response. This includes predefined actions such as isolating affected systems, alerting administrators, and initiating remediation processes.

4. Scalability Solutions: The system architecture will be designed to scale horizontally, allowing for the addition of more nodes to handle increased data loads without compromising performance.

5. Defensive Mechanisms**:** To enhance the resilience of systems against adversarial attacks, techniques such as adversarial training will be used. It involves training models on both legitimate and adversarial examples to improve their robustness.

6. Continuous Learning**:** We will implement a feedback loop where the system continuously learns from new data and adapts to emerging threats is crucial for maintaining effectiveness over time.

3 Method and Implementation

3.1 Methodology

This project adopts an agile methodology to develop and implement an AI-based Intrusion Detection System (AI-IDS) framework, leveraging the publicly available UNSW-NB15 dataset. The iterative nature of AI research, particularly in the dynamic field of cybersecurity, requires a flexible and adaptive approach. Agile principles facilitate the continuous development of models and the effective management of unexpected challenges, ensuring a systematic, well-documented, and ultimately more reproducible research process.

The development lifecycle is structured around iterative sprints, each encompassing planning, implementation, testing, and review phases, thereby enabling continuous feedback and adaptation. Initial sprints focused on the critical stages of data preprocessing and feature engineering specifically tailored for the UNSW-NB15 dataset. Subsequent sprints involved the implementation, training, and rigorous evaluation of individual machine learning classifiers – namely, K-Nearest Neighbour's (KNN) and Random Forest – for both binary (normal vs. attack) and multi-class (specific attack categories) intrusion detection tasks. Later sprints concentrated on integrating these models, exploring potential automated response mechanisms, and considering aspects of scalability and robustness within a simulated environment.

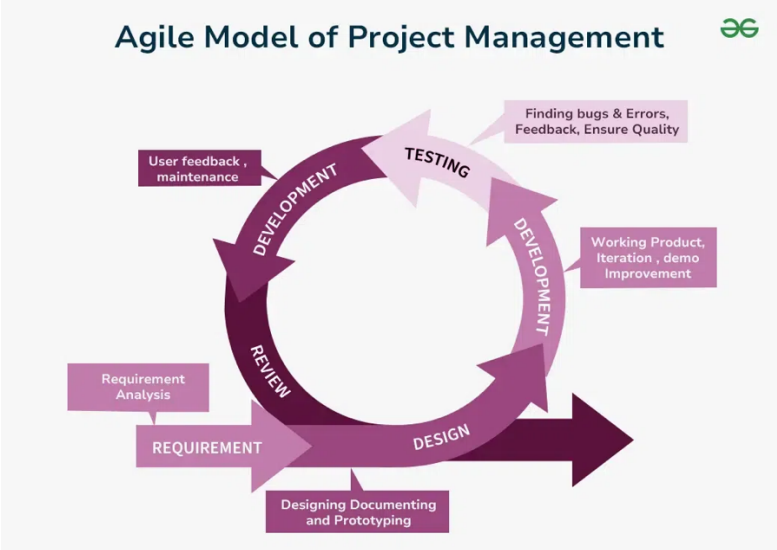


Figure 1.Agile Iterative Sprint Structure. (GeeksforGeeks, 2023)

3.2 Tools and Environment

The following equipment and software were utilised in this research:

Software:

- Python: The programming language used for data preprocessing, model training, and evaluation.
- **Pandas & NumPy:** Essential for efficient data loading, manipulation, and numerical operations on the UNSW-NB15 dataset.
- Scikit-learn: A machine learning library providing tools for classification, regression, clustering, and model selection (Scikit-learn, 2024). Key functionalities used included:
    - KNeighborsClassifier: Implementation of the K-Nearest Neighbour's algorithm.
    - RandomForestClassifier: Implementation of the Random Forest algorithm.
    - train_test_split: Function for splitting the dataset into training and testing sets.
    - cross_val_score: For Cross-validation.
    - OneHotEncoder: For handling categorical features.
    - accuracy_score, precision_score, recall_score, and f1_score: For evaluating model performance.
- Google Colab: A cloud-based platform used for executing the Python code. It provides access to computing resources, including GPUs, which accelerated the model training process. All

3.3 Design and Development

3.3.1 Data Preprocessing and Feature Engineering

Prior to model training, the UNSW-NB15 dataset underwent a series of preprocessing steps to ensure data quality and suitability for machine learning algorithms. These steps included:

3.3.1.1 Data Normalisation

Numerical features were normalised to a common scale using Min-Max scaling to [0, 1] . It prevents features with larger magnitudes from dominating the learning process.

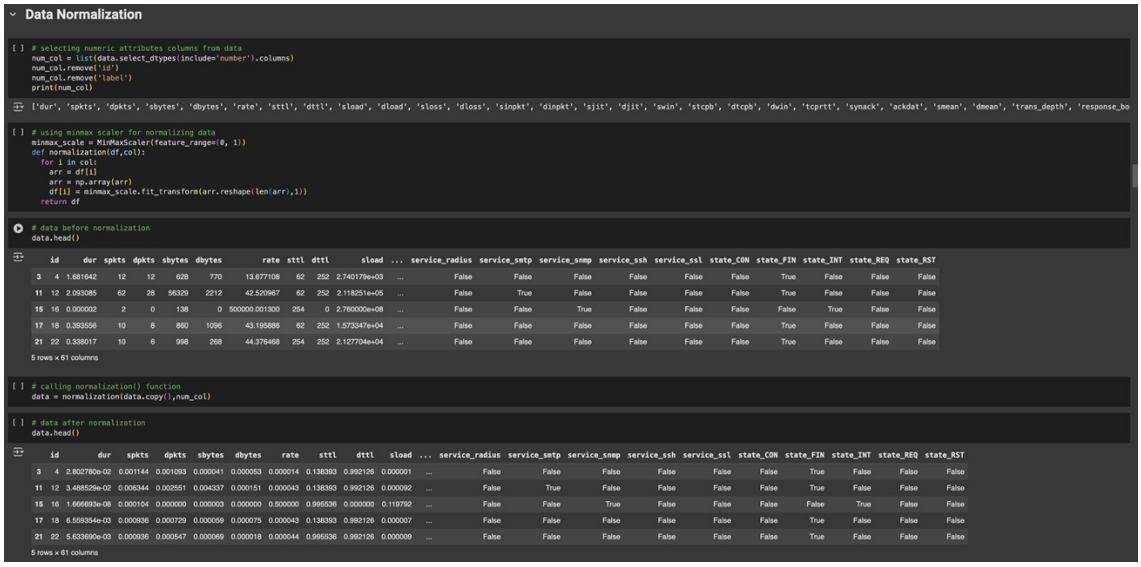


Figure 2. Data Normalisation of the Dataset's numerical values.

3.3.1.2 Categorical Feature Encoding

Categorical features were converted into numerical representations using techniques such as one-hot encoding. This conversion is necessary as most machine learning algorithms require numerical input. It was chosen to avoid imposing an artificial ordinal relationship between the categories.

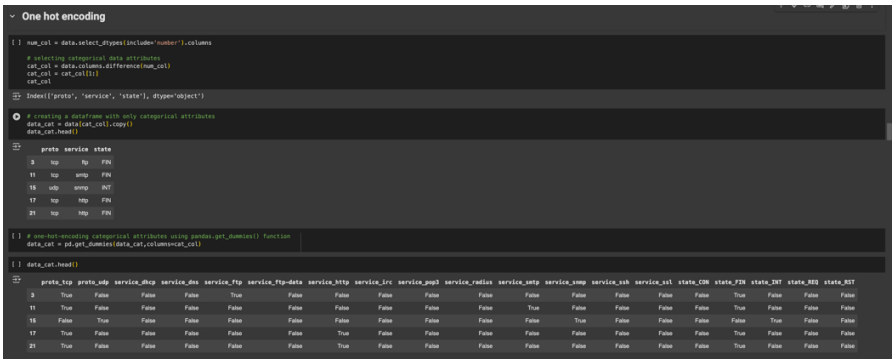


Figure 3. Categorical Encoding - One Hot Encoding.

Following preprocessing, the dataset was prepared for both binary and multi-class classification:

- **Binary Classification:** A new column was created, labelling each instance as either "normal" (0) or "attack" (1), regardless of the specific attack type.

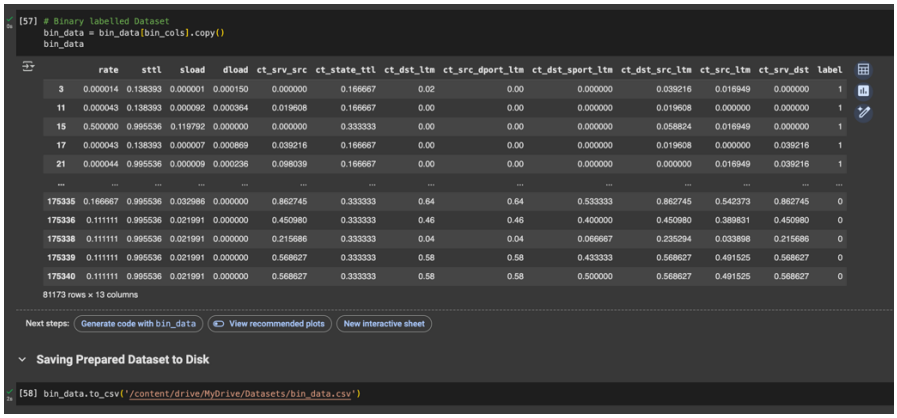


Figure 4. Prepared Dataset for Binary Classification.

- **Multi-class Classification:** The original attack category labels were retained, allowing for the classification of network traffic into one of the nine specific attack types or the "normal" category.

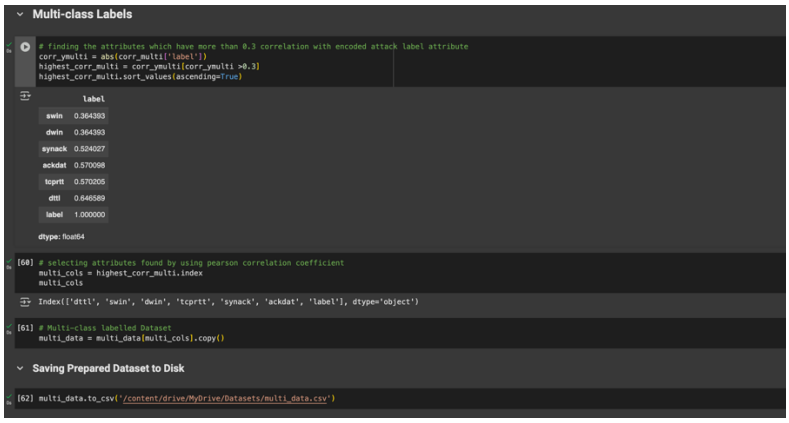


Figure 5. Prepared Dataset for Multi-class Classification.

3.3.1.3 Visualisation

Data visualisation techniques were employed to gain insights into the characteristics of the dataset. This involved generating Pie chart plots for the distribution of the labels on both the binary classification and the multi-class classification.

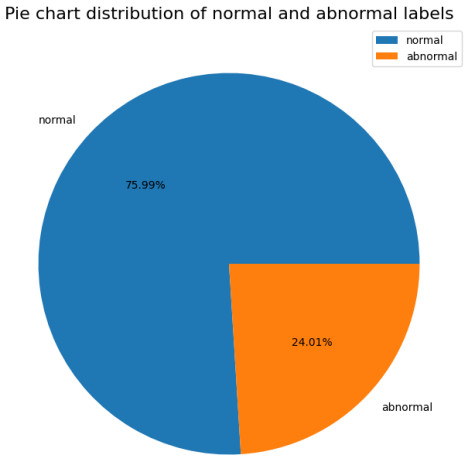


Figure 6. Pie chart Distribution of Binary Labels

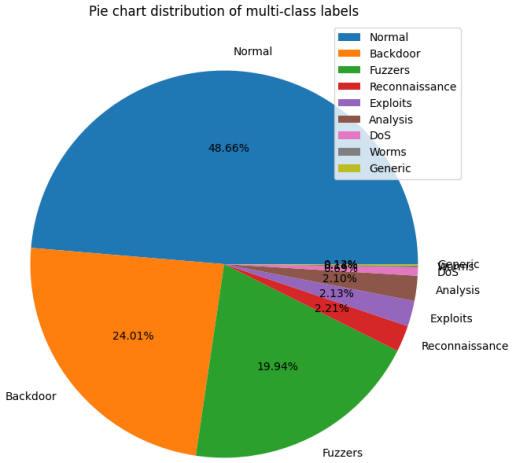


Figure 7. Pie chart Distribution of Multi-Class Labels

These visualisations aided in understanding the dataset's properties and informed subsequent feature selection and model training.

3.3.2 Model Selection

Two machine learning algorithms were selected for this study:

- **K-Nearest Neighbour's (KNN):** KNN is a non-parametric algorithm that classifies an instance based on the majority class of its k nearest neighbour's in the feature space. It was chosen for its simplicity and effectiveness in various classification tasks.
- **Random Forest:** Random Forest is an ensemble learning method that constructs a multitude of decision trees and outputs the class that is the mode of the classes output by individual trees. It was chosen for its robustness, ability to handle high-dimensional data, and capacity to provide feature importance estimates.

3.4 Implementation

3.4.1 Training Process

The training phase involved fitting the selected machine learning algorithms to the training data. This process was carried out separately for both binary and multi-class classification scenarios.

3.4.1.1 Binary Classification Training

- **K-nearest Neighbour**: The KNN model for binary classification involved splitting the dataset into training (80%) and testing (20%) sets to ensure unbiased evaluation. A KNN classifier was then initialised and trained on the training dataset to learn the relationship between features and the binary target variable.



Figure 8. KNN Binary Classification Training

- **Random Forest**: This was trained on the same training dataset. It involved building an ensemble of decision trees, each trained on random subsets of the data and features. By aggregating the predictions of these trees, the Random Forest learns complex patterns for classification. This ensemble approach aims to improve the model's robustness and ability to generalise to unseen data.



Figure 9. Random Forest Binary Classification Training

3.4.1.2 *Multi-class Classification Training*

- **K-nearest Neighbour**: The model underwent a training process to categorise instances into 9 distinct classes: 8 specific attack types and a 'normal' category. The training dataset comprised feature vectors, each associated with a corresponding label indicating its membership within one of these 9 classes. This supervised learning approach enabled the KNN algorithm to learn the relationships between the feature space and the defined attack and normal categories, forming the basis for subsequent classification of unseen data points.



Figure 10. KNN Multi-class Classification Training

- **Random Forest**: Similarly, the Random Forest model constructs and aggregates the predictions of numerous individual decision trees, inherently mitigates the risk of overfitting often associated with single-tree models and enhances predictive accuracy. This approach leverages the strengths of ensemble methods to achieve reliable and generalisable classification performance on complex datasets.



Figure 11. Random Forest Multi-class Classification Training

3.4.2 Cross-Validation

To ensure the generalisability of the models, k-fold cross-validation was employed. The training data was partitioned into k subsets, and the models were trained and evaluated k times, each time using a different subset as the validation set.

4 Evaluation

This project employs a **Quantitative** evaluation methodology. This approach is selected because the core objective of an intrusion detection system is to accurately classify network traffic as either normal or malicious. The effectiveness of such a system can be objectively measured and compared using numerical metrics derived from its predictions on the test dataset. Quantitative evaluation allows for statistical analysis of performance, enabling precise statements about accuracy, reliability, and efficiency.

The selected approach for evaluating the design involved applying the trained machine learning and deep learning models to a held-out test set of network traffic data. The performance was then quantified using standard classification metrics. This method is widely accepted and is a standard practice within the field of intrusion detection research and machine learning model evaluation. It provides a clear, objective, and comparable measure of how well the models generalise to unseen data, which is crucial for assessing their real-world applicability in detecting intrusions. The motivation for this approach lies in its ability to provide concrete, data-driven evidence of the AI-IDS's capabilities, allowing for direct comparison with other detection methods and providing a basis for future improvements.

4.1.1 Evaluation Metrics

Selecting appropriate evaluation metrics is crucial for understanding the true performance of an IDS, especially given the inherent class imbalance (far more normal traffic than attacks) and the different costs associated with different types of errors. The following standard metrics was employed:

- **Accuracy:** The proportion of correctly classified instances (both normal and attack) out of the total number of instances. While intuitive, accuracy can be misleading in datasets with significant class imbalance, which is common in intrusion detection where normal traffic far outweighs attack traffic.

$$Accuracy = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Instances}}$$

- **Precision:** This measures the proportion of instances predicted as attacks that were actually attacks. High precision indicates a low false positive rate, meaning the system generates fewer false alarms. This is important for minimising the workload on security analysts.

$$Precision = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall (Sensitivity):** This measures the proportion of actual attack instances that were correctly identified by the model. High recall indicates a low false negative rate, meaning the system misses fewer actual intrusions. This is often critical in security, as missing an attack can have severe consequences.

$$Recall = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **F1-score:** The harmonic mean of Precision and Recall. The F1-score provides a single score that balances the trade-off between false positives and false negatives, offering a more robust measure than accuracy alone, especially with imbalanced datasets.

$$F1 - score = 2 \text{ x } \frac{\text{Precision x Recall}}{\text{Precision} + \text{Recall}}$$

- **Confusion Matrix:** This is a table that provides a detailed breakdown of predictions versus actual classes. For multi-class classification, it's indispensable for understanding which specific attack types are being correctly identified and which ones are being confused with normal traffic or other attack types. It directly shows the counts of True Positive's, True Negative's, False Positive's, and False Negative's for each class.

These metrics were calculated for both the binary classification task (Normal vs. Attack) and the multi-class classification task (Normal vs. specific attack types). By analysing this set of metrics together, we can gain a clear understanding of each model's strengths and weaknesses in the context of intrusion detection.

4.1.2 Dataset

The UNSW-NB15 dataset was chosen for the evaluation. This dataset is a contemporary benchmark widely used in intrusion detection research, designed to represent a mix of normal network activity and various types of modern attacks. It contains a large volume of network traffic records with 49 features and class labels. The dataset was split into 80% for training and 20% for testing.

The rationale for selecting UNSW-NB15 includes its relevance to current threat landscapes, its comprehensive feature set, and its broad acceptance within the research community, allowing for comparison with results from other studies. The dataset contains nine specific attack categories (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, and Worms) in addition to normal traffic, making it suitable for both binary and multi-class classification tasks.

4.1.3 Baseline Systems

Mahalakshmi G, Uma E, Aroosiya M, and Vinitha M, 2021. Intrusion Detection System Using Convolutional Neural Network on UNSW NB15 Dataset. In: Hemanth, D.J. et al. (eds.) *Advances in Parallel Computing Technologies and Applications*. IOS Press, pp. 1-8. doi: 10.3233/APC210116.

This paper shares a similar goal of using AI techniques, specifically machine learning or deep learning, to build an Intrusion Detection System and evaluate it using the UNSW-NB15 dataset. However, the core models they choose to implement and evaluate as their primary contribution differ significantly.

This project primarily focuses on traditional machine learning algorithms: K-Nearest Neighbour (KNN) and Random Forest. They were evaluated extensively for both binary classification (Normal vs. Attack) and multi-class classification (Normal vs. specific attack types). The project aims to provide results showing both models performing exceptionally well in both the binary and multi-class classifications.

In contrast, the paper's proposed system zeroes in on a specific deep learning model: the Convolutional Neural Network (CNN). CNNs are particularly known for their strength in processing grid-like data, like images, but have been adapted for sequence data and other structures, including network traffic features. The authors trained their CNN on the UNSW-NB15 dataset, specifically focusing on the binary classification task. Their reported result is an accuracy of 93.5% for the CNN model.

The key difference lies in the fundamental approach. This project's implemented work explored established, powerful traditional ML models, while the paper committed to a deep learning architecture. Both approaches have their merits, traditional Machine Learning algorithms can be less computationally expensive and easier to interpret, while deep learning like CNNs has the potential to automatically learn complex features from raw data, potentially capturing more subtle attack

patterns, though this wasn't definitively demonstrated in the reported binary accuracy comparison here. Ultimately, both pieces contribute to the body of work applying AI to the UNSW-NB15 dataset for intrusion detection.

4.1.4 Limitations of the Evaluation

It is important to acknowledge the limitations of this evaluation. Firstly, the findings are based on a single, albeit comprehensive, dataset (UNSW-NB15). Performance might differ on other network traffic distributions or newer datasets reflecting evolved attack techniques. Secondly, this was an offline evaluation. Real-world network traffic is dynamic, and concept drift that changes in traffic patterns over time could degrade model performance without retraining. Additionally, while achieving high metrics, the models still produced false positives and false negatives; the real-world impact and tolerance for these errors need careful consideration in deployment. Finally, computational aspects like training time and prediction latency were not the primary focus of this evaluation but are critical for real-time deployment.

4.1.5 Testing Approach

The practical applicability of these models will be tested by integrating them into a simulated operational environment using **Zeek** for network monitoring and **Wazuh** for security information and event management (SIEM) and data visualization.

Model Deployment in Zeek: The finalised KNN and Random Forest models, trained and tuned as described earlier, will be exported using **joblib**. Then custom Zeek scripts that are capable of loading these models and interfacing with them will be created. Network traffic (either live or from packet captures) will also be monitored and the specific set of features required by our models in near real-time will be extracted

Real-time Intrusion Detection: As Zeek processes network flows, it will feed the extracted features to the loaded models. When the KNN or Random Forest model then predicts an intrusion (classifying traffic as 'attack' or a specific attack type), Zeek will be programmed to generate custom log entries detailing the detection, including which model made the prediction, the predicted attack type, and relevant flow information (like destination IPs and ports).

Forwarding Logs to Wazuh: Zeek will be configured to forward these specialised detection logs, alongside standard Zeek logs, to a Wazuh manager. This allows us to centralise security events and leverage Wazuh's analysis and visualisation capabilities.

Analysis and Visualisation in Wazuh: Within the Wazuh environment (using its Kibana dashboard), the integration enables us to:

- **View Real-time Alerts:** Configure custom rules in Wazuh to specifically parse the incoming Zeek logs generated by our models. When a rule matches a model's intrusion prediction, Wazuh will trigger an alert, which will appear immediately in the security events dashboard.

- **Query and Investigate:** The Wazuh/Kibana interface will be used to easily query the indexed Zeek data. This allows us to search for specific types of attacks detected by the models, filter events based on IP addresses or time ranges, and drill down into the details logged by Zeek for incidents flagged by the AI models.

- **Visualise Detection Patterns:** Visualisations and dashboards within Wazuh/Kibana will be created to gain insights from the models' activity. This includes generating graphs showing the frequency of detected attacks over time, pie charts illustrating the distribution of different attack types flagged by KNN versus Random Forest (similar to what is shown above in Figure and Figure), and tables summarising the most frequent sources of detected malicious activity.

This integrated testing setup aims to provide valuable insights into how the models perform under more realistic, continuous data flow conditions and demonstrate how their detections could be seamlessly incorporated into a standard security monitoring and visualisation platform like Wazuh for actionable intelligence.

4.2 Results

This section presents the results obtained from evaluating the KNN and Random Forest models on the held-out UNSW-NB15 test set. The performance of each classifier is quantified using the evaluation metrics calculated from the confusion matrix. The results are presented separately for the binary classification task (Normal vs. Attack) and the more multi-class classification task (Normal vs. specific attack types). These measurements provide crucial insights into the effectiveness and limitations of each model for intrusion detection within this dataset based on offline evaluation.

4.2.1 Binary Classification Performance

The evaluation focused on the fundamental task of distinguishing between (Normal) network traffic and malicious (Attack) traffic, regardless of the specific attack vector. Both KNN and Random Forest models were assessed on their ability to perform this binary classification on the UNSW-NB15 test set.

Metrics	**K-Nearest Neighbour**	**Random Forest**
Accuracy	98.1%	98.4%
Precision	0.98	0.99
Recall	0.99	0.99
F1-Score	0.99	0.99

Table 1. Binary Classification Performance on Dataset Test – ATTACK

Metrics	**K-Nearest Neighbour**	**Random Forest**
Accuracy	98.1%	98.4%
Precision	0.97	0.98
Recall	0.95	0.96
F1-Score	0.96	0.97

Table 2. Binary Classification Performance on Dataset Test – NORMAL

The overall performance metrics for the 'Attack' class are summarised in Table 1, while the corresponding metrics for the 'Normal' class are shown in Table 2. Looking first at overall Accuracy, the Random Forest model achieved a slightly higher accuracy of 98.4% compared to KNN's 98.1%. While both figures are remarkably high, accuracy alone can sometimes be misleading in security

datasets due to inherent imbalances, even in a binary split where one class might still be more prevalent. Therefore, examining Precision, Recall, and F1-score provides a more subtle view.

For the critical 'Attack' class (Table 1), Random Forest again demonstrates a marginal edge across all metrics. KNN also performs exceptionally well, with Precision of 0.98 and Recall and F1-Scores of 0.99. High precision for the 'Attack' class on the KNN is vital as it indicates a low rate of false positives which classifies normal traffic as an attack. A 0.99 precision means that when Random Forest flags traffic as an attack, it is correct 99% of the time. High recall, also 0.99 for both, signifies a low rate of false negatives which fails to detect an actual attack on the KNN and also signifies that Random Forest successfully identified 99% of actual attacks in the test set. The high F1-scores reflect the strong balance between precision and recall.

Considering the 'Normal' class performance (Table 2), Random Forest again leads slightly with Precision (0.98), Recall (0.96), and F1-Score (0.97), compared to KNN's 0.97 Precision, 0.95 Recall, and 0.96 F1-Score. High precision for this classification means that when the model predicts traffic is normal, it is very likely correct, minimising the false positives on attack traffic. High recall for this classification means the model correctly identifies most normal traffic, minimising the false negatives on normal traffic.

4.2.2 Multi-class Classification Performance

Moving to the more challenging task of multi-class classification, the models were evaluated on their ability to classify network traffic into nine categories: a normal attack and 8 specific attack types present in UNSW-NB15 and also the weighted average on the metrics. This provided a deeper understanding of the model's capability to not just detect an intrusion but also to potentially identify the nature of the threat.

Metrics	**K-Nearest Neighbour**	**Random Forest**
Accuracy	98.1%	88.7%
Weighted Precision	0.86	0.86
Weighted Recall	0.88	0.89
Weighted F1	0.86	0.87

Table 3. Multi-Class Classification Performance on Dataset Test Set - Weighted Avg.

Metrics	**Precision**	**Recall**	**F1-Score**
Normal	0.98	0.91	0.94
Analysis	0.26	0.22	0.24
Backdoor	0.00	0.00	0.00
DoS	0.11	0.04	0.06
Exploits	0.69	0.91	0.78
Fuzzers	0.17	0.04	0.06
Generic	0.98	0.99	0.99
Reconnaissance	0.02	0.00	0.01
Worms	0.00	0.00	0.00

Table 4. Multi-Class Categorical Classification Performance- KNN

Metrics	Precision	Recall	F1-Score
Normal	0.97	0.92	0.94
Analysis	0.67	0.44	0.53
Backdoor	0.12	0.06	0.08
DoS	0.17	0.06	0.09
Exploits	0.70	0.90	0.79
Fuzzers	0.29	0.09	0.14
Generic	0.98	0.99	0.99
Reconnaissance	0.12	0.03	0.05
Worms	0.00	0.00	0.00

Table 5. Multi-Class Categorical Classification Performance- Random Forest

Table 3 presents the overall accuracy and weighted average metrics for both models on the multi-class task. Interestingly, K-Nearest Neighbours achieved a significantly higher overall Accuracy of 98.1% compared to Random Forest's 88.7%. This initial finding might seem unreasonable given Random Forest's stronger performance in the binary task and its general robustness. However, interpreting multi-class results in datasets with significant class imbalance requires careful consideration of weighted average metrics and per-class performance. The weighted average Precision, Recall, and F1-Score provide a view that accounts for the number of instances in each class. In this case, the weighted averages are much closer, with Random Forest holding a slight edge in weighted average Recall and F1-Score. This suggests that while KNN's high overall accuracy might be heavily influenced by correctly classifying the majority 'Normal' class, Random Forest might be performing better when considering the models' performance across all classes proportional to their presence in the dataset.

To truly understand the multi-class performance, we must examine the metrics for each individual class, as shown in Table 4 (KNN) and Table 5 (Random Forest). A clear trend occurs immediately from these tables: both models perform exceptionally well on the 'Normal' and 'Generic' classes, achieving very high Precision, Recall, and F1-scores (often exceeding 0.90, and close to 0.98-0.99 for 'Generic'). This indicates that both algorithms are highly effective at identifying traffic and instances of the most frequent attack type.

However, performance drops significantly for most of the other attack categories, particularly those with fewer instances in the dataset. For KNN (Table 4), classes like 'Backdoor' and 'Worms' have F1-scores of 0.00, indicating a complete failure to correctly identify any instances of these attack types which is likely due to zero recall or zero precision for those classes. 'Analysis', 'DoS', 'Fuzzers', and 'Reconnaissance' also show very low F1-scores (0.24, 0.06, 0.06, and 0.01 respectively), with corresponding low precision and recall values. This reveals a significant challenge for KNN in distinguishing and accurately classifying these less represented or potentially more subtle attack patterns.

Random Forest (Table 5) also struggles with the rare attack types, with 'Worms' again showing an F1-score of 0.00. However, Random Forest demonstrates better performance on several other challenging categories compared to KNN. For instance, its F1-score for 'Analysis' is 0.53 (vs KNN's 0.24), for 'Backdoor' is 0.08 (vs KNN's 0.00), for 'DoS' is 0.09 (vs KNN's 0.06), for 'Fuzzers' is 0.14 (vs KNN's 0.06), and for 'Reconnaissance' is 0.05 (vs KNN's 0.01). While these F1-scores are still relatively low, they represent a notable improvement over KNN and indicate that Random Forest is marginally

more successful at capturing the patterns associated with these less frequent intrusions. The 'Exploits' class is classified with moderate success by both models, with F1-scores around 0.78-0.79.

The high overall accuracy of KNN in the multi-class task (Table 3) is thus likely inflated by its excellent performance on the dominant 'Normal' class, covering its poor ability to detect many specific attack types. Random Forest, while having lower overall accuracy, exhibits better performance on the weighted average metrics and, critically, shows a greater capacity to identify instances from the less frequent attack categories, as seen in table 5.

4.2.3 Comparative Summary

Comparing the performance across both tasks, the Random Forest model generally emerges as the stronger classifier for this intrusion detection. In the binary classification, it achieved slightly higher metrics consistently. In the more complex multi-class scenario, despite KNN's higher raw accuracy, Random Forest's superior weighted average metrics and its better performance on individual low-frequency attack classes indicate that it is more effective at the task of identifying different types of intrusions when considering the dataset's imbalance.

This difference in performance likely stems from their fundamental approaches. Random Forest, as an ensemble method, combines the predictions of multiple decision trees, reducing variance and overfitting, and indirectly performing feature selection which can be beneficial in high-dimensional data like network traffic features. KNN’s reliance on distance metrics to find nearest neighbours, can be sensitive to the scale and distribution of features and may struggle more in high-dimensional spaces or with complex decision boundaries that separate closely related attack types or distinguish attacks from normal traffic.

For example, examining some misclassified instances from the test set, we can observe cases where a 'Fuzzers' attack, exhibiting subtle deviations from normal behaviour, was correctly flagged by Random Forest but misclassified as 'Normal' by KNN. This could be attributed to Random Forest's ability to learn more complex, axis-aligned decision boundaries across multiple features within its ensemble, whereas KNN might be misled by the overall Euclidean distance to a cluster of normal instances in the high-dimensional space. Conversely, in the binary task, instances where KNN misclassified a 'Normal' flow as 'Attack' might occur if that normal flow had feature values placing it unusually close to a cluster of attack instances.

4.2.4 Challenges in Practical Testing Implementation

As outlined in the planned testing approach, a crucial next step after offline model evaluation was to integrate the trained models into a simulated operational environment using Zeek for network monitoring and Wazuh for SIEM and visualisation. However, during the implementation phase of this integrated setup, a significant technical impediment was encountered that prevented the full execution of the planned practical testing.

Unfortunately, attempts to establish a stable API connection between the Wazuh Dashboard and the Wazuh Manager were unsuccessful. This persistent connection issue, despite troubleshooting efforts within the project timeline, meant that logs forwarded from Zeek, including those generated by the AI models' detections, could not be reliably visualised, queried, or analysed within the Wazuh interface as intended. This broke the chain of the planned testing methodology, which relied on Wazuh's capabilities to demonstrate the models' performance in a near-real-time, actionable security monitoring context.

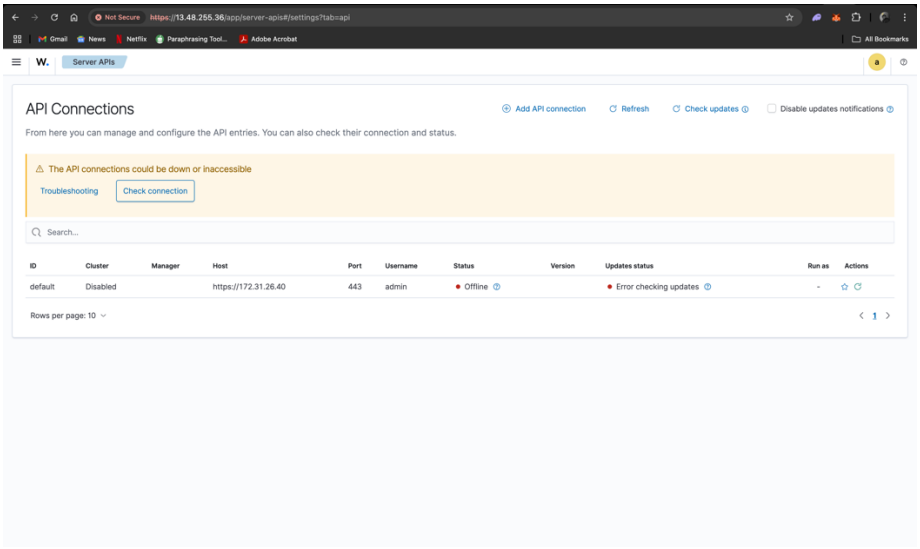


Figure 12. Unsuccessful API Connection on Wazuh's Dashboard

4.3 Discussion

The consistently high performance of both models in the binary classification task, correctly distinguishing between normal and attack traffic with accuracy, precision, recall, and F1-scores in the high 90s is encouraging. This suggests that the preprocessing steps and the selected feature set effectively capture the differences between normal and malicious network activities present in this dataset. This level of performance for binary classification on UNSW-NB15 is generally consistent with outcomes reported in much of the existing literature, confirming that standard machine learning techniques can achieve high overall detection rates on well-represented classes within this benchmark dataset.

However, the transition to the multi-class classification task exposed a significant challenge, particularly regarding the detection of less frequent attack types. While both models maintained excellent performance on the 'Normal' and 'Generic' categories, performance dropped drastically for classes like 'Analysis', 'Backdoor', 'DoS', 'Fuzzers', 'Reconnaissance', and 'Worms'. The near zero F1-scores for some of these classes highlight a common challenge in intrusion detection: handling class imbalance. This phenomenon is widely discussed in literature concerning IDS datasets; models tend to favour the majority class and struggle to learn the subtle patterns of rare events, precisely the kind of stealthy or novel attacks that are critical to detect in a real-world scenario.

Comparing the two models in the multi-class task reveals Random Forest's relative strength, particularly evident in the weighted average metrics and its improved, although still limited, performance on several rare attack types compared to KNN . This difference likely stems from the basic properties of the algorithms. Random Forest, as an ensemble of decision trees, is more robust to noisy data and can capture complex, non-linear relationships within the features that might define specific attack variations. Its tree-based nature also lends itself to handling interactions between features implicitly. In contrast, KNN's reliance on distance metrics in a high-dimensional feature space, even after normalisation, can be vulnerable to the "curse of dimensionality" where the concept of "nearest" becomes less meaningful, making it harder to set boundaries between closely related classes or detect unusual instances of rare attacks. Random Forest's inherent ability to weigh feature importance during tree construction may also contribute to its better performance on discriminative features for different attack types.

It's important to acknowledge the limitation that these findings are based solely on offline evaluation on a static dataset. As discussed earlier, the inability to implement and test the models within the planned Zeek and Wazuh environment due to an API connectivity issue means we could not validate their performance in a dynamic, near-real-time setting. Real-world network traffic involves concept drift changes in traffic patterns and attack methodologies over time which can degrade model performance without continuous retraining or adaptive strategies. The practical challenges of integrating models into live monitoring systems and managing the operational aspects (like false alarm rates impacting analyst workload) remain unexplored in this study.

5 Conclusions

The project aimed to explore how AI could enhance intrusion detection to better defend against sophisticated cyber threats. It was concluded that standard machine learning techniques, specifically K-Nearest Neighbour and Random Forest, demonstrate a strong capability in distinguishing between normal and malicious network traffic during binary classification on the UNSW-NB15 dataset, confirming their potential for basic threat identification.

Furthermore, practical challenges prevented the crucial real-world validation of these models within a simulated operational environment. While progress was made in evaluating individual algorithms against the project objectives, the goal of developing and testing a performance-enhanced hybrid model remains a vital area for future research to fully address the complexity of modern intrusion detection.

6 Recommendations for future work

Building upon this work, future research should primarily focus on enhancing the detection of rare and novel attack types, which proved challenging for the evaluated models. Exploring advanced techniques for handling class imbalance, such as data augmentation or cost-sensitive learning, is therefore highly recommended as well.

Furthermore, the development and rigorous testing of hybrid AI models that combine the strengths of different algorithms holds significant promise for improved accuracy and should be pursued, directly addressing an objective from this project that could not be fully realised. Overcoming the practical implementation challenges encountered, ensuring seamless integration with security monitoring platforms like Zeek and Wazuh, is essential for transitioning from theoretical evaluation to real-world applicability. Finally, assessing model performance in dynamic, real-time environments subject to concept drift and investigating adversarial resilience will be vital steps towards building truly robust and deployable AI-based intrusion detection systems for the future.

7 References

1. Ahmed, U., Jiangbin, Z., Almogren, A., Khan, S., Sadiq, M., Altameem, A. and Rehman, A. (2024). Explainable AI-based innovative hybrid ensemble model for intrusion detection. Journal of Cloud Computing, 13(1). doi:https://doi.org/10.1186/s13677-024-00712-x.

2. Alghamdi, M.I. (2022). A Hybrid Model for Intrusion Detection in IoT Applications. Wireless Communications and Mobile Computing, 2022(01), pp.1–9. doi:https://doi.org/10.1155/2022/4553502.

3. Althar, R.R., Samanta, D., Purushotham, S., Sengar, S.S. and Hewage, C. (2023). Design and Development of Artificial Intelligence Knowledge Processing System for Optimizing Security of Software System. SN Computer Science, 4(4). doi:https://doi.org/10.1007/s42979-023-01785-2.

4. Armstrong, P. (2021). IoT devices: An unfortified attack vector | CXO Revolutionaries. [online] Zscaler.com. Available at: https://www.zscaler.com/cxorevolutionaries/insights/iot-devices-unfortified-attack-vector [Accessed 19 Mar. 2025].

5. Bishop, M. (2018) Computer Security: Art and Science. 2nd edn. Addison-Wesley Professional.

6. Chirra, D. (2023). Towards an AI-Driven Automated Cybersecurity Incident Response System. International Journal of Advanced Engineering Technologies and Innovations, 01(01)

7. CYBER SNIPER (2024). Why Traditional Security Models Are No Longer Enough - CyberSniper. [online] CyberSniper. Available at: https://www.cybersniper.ai/why-traditional-security-models-are-no-longer-enough/.
8. Danyil, V., Valerii, H., Oleksandr, B., Volodymyr, P., Oleksandr, K., Mykola, K., Volodymyr, S. and Serhii, S. (2023) 'Real-Time Ransomware Detection System Integrating eBPF, Machine Learning, and Natural Language Processing', in 2023 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Antenna Theory (RSSI), pp. 103–108.

9. Diana, L., Dini, P. and Paolini, D. (2025). Overview on Intrusion Detection Systems for Computers Networking Security. Computers, 14(3), p.87. doi:https://doi.org/10.3390/computers14030087.

10. Diana, S., Nurfauzi, N.I., Akbar, M.S., Adli, M.A. and Luthfi, R. (2025) 'The Role of Artificial Intelligence in Enhancing Intrusion Detection Systems', International Journal of Advanced Computer Science and Applications, 16(1).

11. Etheridge, E. (2024). Cyber security measures: Secure your business with digital signatures. Dataguard.co.uk, [online] 1(1). Available at: https://www.dataguard.co.uk/blog/what-is-a-digital-signature-and-how-it-can-secure-your-business/#which-industries-need-digital-signatures-most.

12. Faisal, S. (2024). Artificial Intelligence based Intrusion Detection Systems for IoT Devices. [online] Available at: https://www.researchgate.net/publication/384321903_Artificial_Intelligence_based_Intrusion_Detection_Systems_for_IoT_Devices.

13. Farooq, M., Safdar, M. and Tehseen, S. (2018) ‘An Empirical Study of Machine Learning Techniques for Intrusion Detection Systems’, International Journal of Computer Science and Security (IJCSS), 12(1), pp. 1–10.

14. Farzaan, M., Ghanem, M., El-Hajjar, A. and Ratnayake, D. (2024). AI-Enabled System for Efficient and Effective Cyber Incident Detection and Response in Cloud Environments.

15. geeksforgeeks (2023). What is Sprint? [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/what-is-a-sprint-in-agile/.

16. Herrero, Á., Navarro, M., Corchado, E. and Julián, V. (2013). RT-MOVICAB-IDS: Addressing real-time intrusion detection. Future Generation Computer Systems, 29(1), pp.250–261. doi:https://doi.org/10.1016/j.future.2010.12.017.

17. Ibitoye, O., Shafiq, O. and Matrawy, A. (2019). Analyzing Adversarial Attacks Against Deep Learning for Intrusion Detection in IoT Networks.

18. Jamshed, M., Lee, J., Moon, S., Yun, I., Kim, D., Lee, S., Yi, Y. and Park, K. (2012). Kargus: A Highly-scalable Software-based Intrusion Detection System General Terms.

19. Kahani, M., Mighan, S.M., Siami, N. and Khanli, L.M. (2020) ‘A Novel Scalable Intrusion Detection System Based on Deep Learning in Big Data Environment’, International Journal of Advanced Computer Science and Applications, 11(1).

20. Karatas, G., Demir, O. and Koray Sahingoz, O. (2018). Deep Learning in Intrusion Detection Systems.

21. Mahalakshm G, Uma E, Aroosiya M and Vinitha M (2021). Intrusion Detection System Using Convolutional Neural Network on UNSW NB15 Dataset. Advances in Parallel Computing, 1(1). doi:https://doi.org/10.3233/apc210116.

22. Mighan, S.N. and Kahani, M. (2020). A novel scalable intrusion detection system based on deep learning. International Journal of Information Security, 20(3), pp.387–403. doi:https://doi.org/10.1007/s10207-020-00508-5.

23. Mohale, V.Z. and Obagbuwa, I.C. (2025). A systematic review on the integration of explainable artificial intelligence in intrusion detection systems to enhancing transparency and interpretability in cybersecurity. Frontiers in Artificial Intelligence, 8. doi:https://doi.org/10.3389/frai.2025.1526221.

24. Mohammed I. Alghamdi (2022) ‘PO-CFNN: An Enhanced Intrusion Detection System for IoT Environments Using Political Optimizer and Cascade Forward Neural Network’, Electronics, 11(15), p. 2358.

25. Patel, H. (2024). Feature Engineering Explained | Built In. [online] builtin.com. Available at: https://builtin.com/articles/feature-engineering.

26. Pawlicki, M., Choraś, M. and Kozik, R. (2020). Defending network intrusion detection systems against adversarial evasion attacks. Future Generation Computer Systems, 110, pp.148–154. doi:https://doi.org/10.1016/j.future.2020.04.013.

27. Pawlick, S. (2020) Adversarial Machine Learning for Cyber Security. Springer.

28. Protasiewicz, J. (2025). Python AI: Why Is Python So Good for Machine Learning? [online] www.netguru.com. Available at: https://www.netguru.com/blog/python-machine-learning.

29. Ramyavarshini, P., Sriram, G.K., Rajasekaran, U. and Malini, A. (2022). Explainable AI for Intrusion Detection Systems. 2022 5th International Conference on Contemporary Computing and Informatics (IC3I), pp.1563–1567.doi:https://doi.org/10.1109/ic3i56241.2022.10073356.

30. R, V., A, P.C. and M, V. (2024). A Comprehensive Analysis of Intrusion Detection System using Machine Learning and Deep Learning Algorithms. 2024 International Conference on Intelligent Algorithms for Computational Intelligence Systems (IACIS), pp.1–5. doi:https://doi.org/10.1109/iacis61494.2024.10721636.

31. Saha, K. and Pritom, R. (2023). A Comparative Analysis of ML Techniques for Real-Time Monitoring and Incident Response. doi:https://doi.org/10.5281/zenodo.8060932.

32. Saravanan, T., S, D. and P, S. (2023). ScienceDirect. Advanced EGAN-IDS Framework for Resilience against Adversarial Attacks Using Multi-headed Attention Module Adversarial Attacks Using Multi-headed Attention Module, 01(01).

33. doi:https://doi.org/10.1016/j.procs.2023.12.075.

34. Scikit-learn (2024). scikit-learn: Machine Learning in Python. [online] Scikit-learn.org. Available at: https://scikit-learn.org/stable/.
35. Sriram, V., Ramyavarshini, T. and Geetha, S. (2022) 'Interpretable AI for Anomaly-Based Intrusion Detection Systems', International Journal of Engineering Trends and Technology, 70(12), pp. 1–7.

36. Talla, G. (2023). Process of Machine Learning: Data, Model and Learning. [online] Medium. Available at: https://giridhartalla.medium.com/process-of-machine-learning-data-model-and-learning-d504715eceab [Accessed 19 Mar. 2025].

37. Thakkar, A. and Lohiya, R. (2021). A survey on intrusion detection system: feature selection, model, performance measures, application perspective, challenges, and future research directions. Artificial Intelligence Review, 55(1), pp.453–563. doi:https://doi.org/10.1007/s10462-021-10037-9.

38. Uhm, Y. and Pak, W. (2021). Service-Aware Two-Level Partitioning for Machine Learning-Based Network Intrusion Detection With High Performance and High Scalability. IEEE Access, 9(01), pp.6608–6622. doi:https://doi.org/10.1109/access.2020.3048900.

39. Varaprasad, G., Vani, T., Kumar, P.S., Reddy, B.M., Prasad, A.S. and Srujana, M. (2024) 'Comparison of Machine Learning and Deep Learning Models for Intrusion Detection', International Journal of Intelligent Systems and Applications in Engineering, 12(1s), pp. 658–666.

40. West, D. (2024). Sprint Planning. [online] Atlassian. Available at: https://www.atlassian.com/agile/scrum/sprint-planning.

41. Zhang, S. and Ahmad, N. (2020). Integrating AI with Infrastructure Protection: a Framework for Advanced Threat Detection in Cloud and Information Security. doi:https://doi.org/10.13140/RG.2.2.15716.18566.

42. Zhuravchak, D. and Dudykevych, V. (2023). Real-Time Ransomware Detection by Using eBPF and Natural Language Processing and Machine Learning. 2023 IEEE 5th International Conference on Advanced Information and Communication Technologies (AICT), pp.1–4. doi:https://doi.org/10.1109/aict61584.2023.10452697.

8 Appendices

8.1 Gantt Chart

